

Constrained Horn Clauses and their Applications

Lecture 3

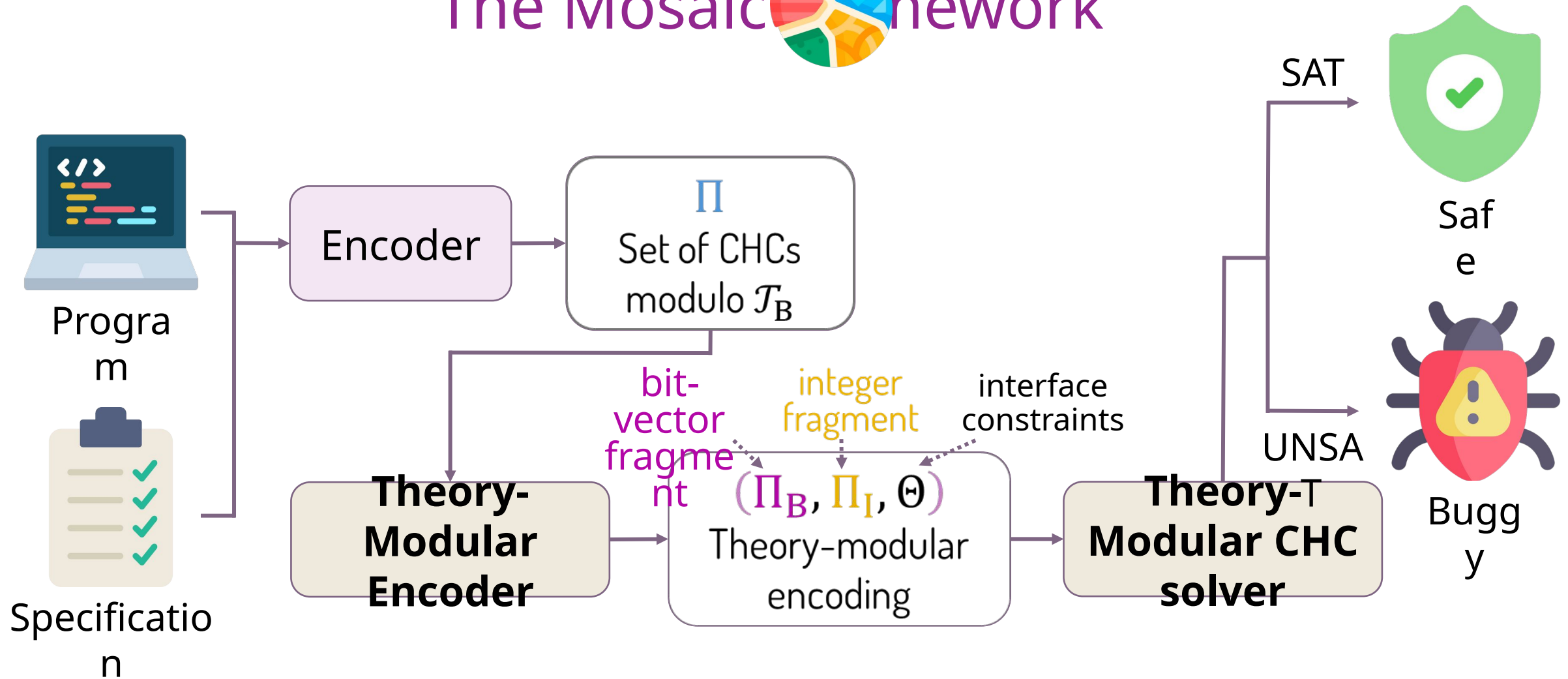
Orna Grumberg, Technion

Bit-Precise CHC Satisfiability Using Theory- Modular Reasoning

Omer Rappoport, Orna Grumberg, Yakir
Vizel

ATVA 2026

The Mosaic network



Encoding, step 1: partition

Π :

$$\langle 0 \rangle \quad (x >_s^B 0000 \wedge a = 0000 \wedge b = 0000) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad (p(x, a, b) \wedge a <_s^B x) \rightarrow p(x, a +^B 0001, b -^B 0001)$$

$$\langle 2 \rangle \quad (p(x, a, b) \wedge \neg(a <_s^B x)) \rightarrow q(a, b)$$

$$\langle 3 \rangle \quad q(a, b) \rightarrow ((a \oplus^B b) <_s^B 0000)$$

Encoding, step 1: partition

→ **integer fragment**

$$\langle 0 \rangle \quad (x >_s^B 0000 \wedge a = 0000 \wedge b = 0000) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad (p(x, a, b) \wedge a <_s^B x) \rightarrow p(x, a +^B 0001, b -^B 0001)$$

$$\langle 2 \rangle \quad (p(x, a, b) \wedge \neg(a <_s^B x)) \rightarrow q(a, b)$$

→ **bit-vector fragment**

$$\langle 3 \rangle \quad q(a, b) \rightarrow ((a \oplus^B b) <_s^B 0000)$$

Heuristic: clauses whose function applications are purely arithmetic → integer fragment

Encoding, step 2: the bit-vector fragment

→ integer fragment

$$\langle 0 \rangle \quad (x >_s^B 0000 \wedge a = 0000 \wedge b = 0000) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad (p(x, a, b) \wedge a <_s^B x) \rightarrow p(x, a +^B 0001, b -^B 0001)$$

$$\langle 2 \rangle \quad (p(x, a, b) \wedge \neg(a <_s^B x)) \rightarrow q(a, b)$$

Π_B :

$$\langle 3 \rangle \quad q^B(a, b) \rightarrow ((a \oplus^B b) <_s^B 0000)$$

i. Predicate
renaming

Encoding, step 3: the integer fragment

Π_I :

$$\begin{aligned} \langle 0 \rangle \quad & \overbrace{(x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0)}^{x' \upharpoonright_B} \rightarrow p^I(x', a', b') \\ \langle 1 \rangle \quad & (p^I(x', a', b') \wedge a' < x') \rightarrow \\ & p^I\left(x', \text{uts}_4((a' + 1) \bmod 16), \text{uts}_4((b' - 1) \bmod 16)\right) \\ \langle 2 \rangle \quad & (p^I(x', a', b') \wedge \neg(a' < x')) \rightarrow q^I(a', b') \end{aligned}$$

Π_B :

$$\langle 3 \rangle \quad q^B(a, b) \rightarrow \left((a \oplus^B b) <_s^B 0000 \right)$$

- i. Predicate renaming
- ii. Variable renaming
- iii. Constraint translations
- iv. Domain restrictions

Encoding, step 3: the integer fragment

Π_I :

- i. Predicate renaming
- ii. Variable renaming
- iii. Constraint translations
- iv. Domain restrictions

$$\begin{aligned} \langle 0 \rangle \quad & \overbrace{(x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0)}^{x' \upharpoonright_B} \rightarrow p^I(x', a', b') \\ \langle 1 \rangle \quad & (p^I(x', a', b') \wedge a' < x') \rightarrow \\ & p^I\left(x', \text{uts}_4((a' + 1) \bmod 16), \text{uts}_4((b' - 1) \bmod 16)\right) \\ \langle 2 \rangle \quad & (p^I(x', a', b') \wedge \neg(a' < x')) \rightarrow q^I(a', b') \end{aligned}$$

Π_B :

$$\langle 3 \rangle \quad q^B(a, b) \rightarrow ((a \oplus^B b) <_s^B 0000)$$

Optimization: eliminate modulo and uts operations where overflow is impossible

Encoding, step 4: interface constraints

Π_I :

$$\langle 0 \rangle \quad (x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0) \rightarrow p^I(x', a', b')$$

$$\langle 1 \rangle \quad (p^I(x', a', b') \wedge a' < x') \rightarrow p^I(x', a' + 1, b' - 1)$$

$$\langle 2 \rangle \quad (p^I(x', a', b') \wedge \neg(a' < x')) \rightarrow q^I(a', b')$$

Π_B :

$$\langle 3 \rangle \quad q^B(a, b) \rightarrow ((a \oplus^B b) <_s^B 0000)$$

$$\Theta = \{q^I \rightsquigarrow q^B\}$$

Semantics: derivations across fragments

Π_I :

$\langle 0 \rangle \quad (x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0) \rightarrow p^I(x', a', b')$

$\langle 1 \rangle \quad (p^I(x', a', b') \wedge a' < x') \rightarrow p^I(x', a' + 1, b' - 1)$

$\langle 2 \rangle \quad (p^I(x', a', b') \wedge \neg(a' < x')) \rightarrow q^I(a', b')$

Π_B :

$\langle 3 \rangle \quad q^B(a, b) \rightarrow ((a \oplus^B b) <_s^B 0000)$

$\Theta = \{q^I \rightsquigarrow q^B\}$

local
 $p^I(1, 0, 0)$
local
 $p^I(1, 1, -1)$
local
 $q^I(1, -1)$
external
 $q^B(0001, 1111)$

Derivation

$0001 \oplus^B 1111 = 1110 <_s^B 0000$

**Query
Check**



Semantics: theory-modular solutions

Π_I :

$$\langle 0 \rangle \quad (x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0) \rightarrow p^I(x', a', b')$$

$$\langle 1 \rangle \quad (p^I(x', a', b') \wedge a' < x') \rightarrow p^I(x', a' + 1, b' - 1)$$

$$\langle 2 \rangle \quad (p^I(x', a', b') \wedge \neg(a' < x')) \rightarrow q^I(a', b')$$

$$\sigma_I(p^I(x', a', b')) := x' > 0 \wedge a' = -b'$$

$$\sigma_I(q^I(a', b')) := a' > 0 \wedge b' < 0$$

$$\sigma_B(q^B(a, b)) := a >_s^B 0000 \wedge b <_s^B 0000$$

Π_B :

$$\langle 3 \rangle \quad q^B(a, b) \rightarrow ((a \oplus^B b) <_s^B 0000)$$

Satisfying Interpretation



$$\Theta = \{q^I \rightsquigarrow q^B\}$$

1. **Local satisfaction:** $\sigma_B \models_{\mathcal{T}_B} \Pi_B$ and $\sigma_I \models_{\mathcal{T}_I} \Pi_I$

2. **Interface constraint satisfaction:** $\models_{\mathcal{T}_B} \tau_{I \rightsquigarrow B}(\sigma_I(q^I)) \rightarrow \sigma_B(q^B)$

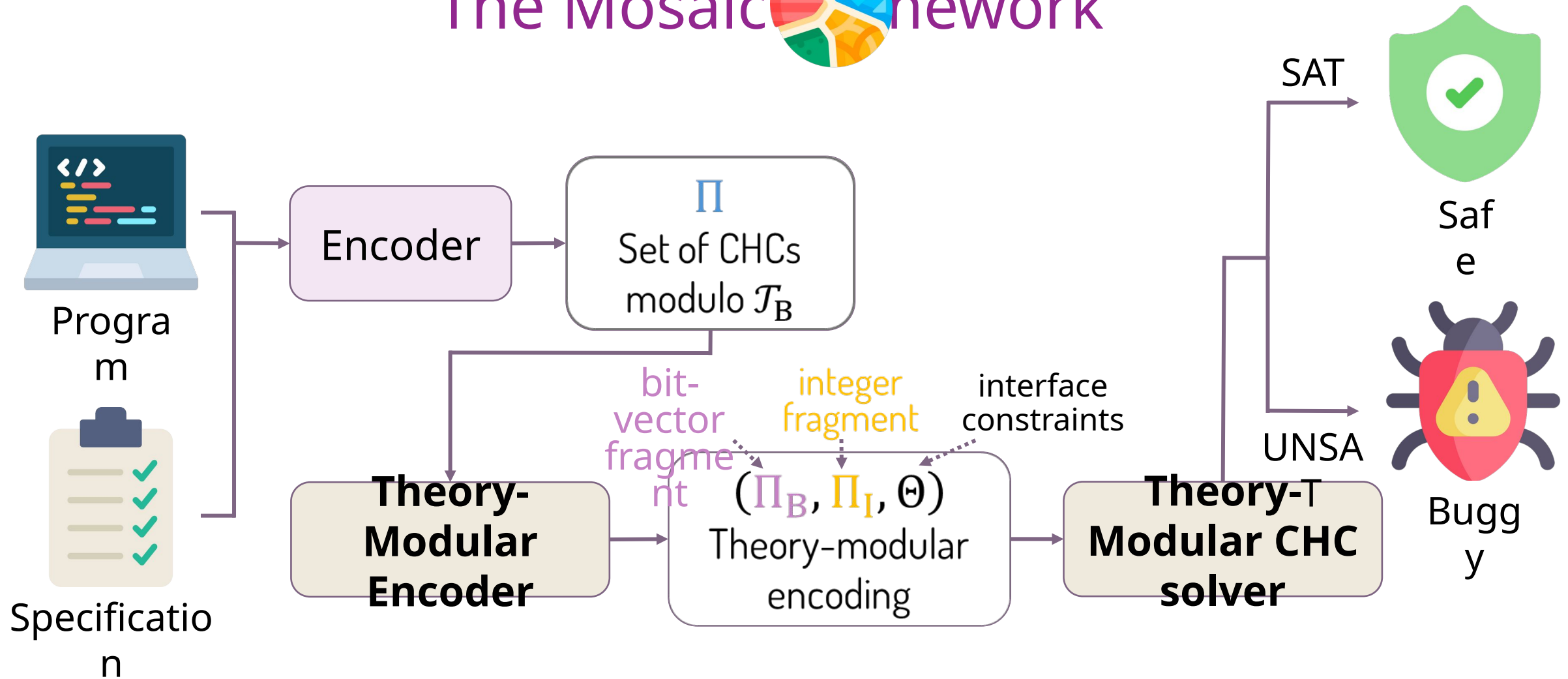
Correctness

Let Π be a set of CHCs modulo $\mathcal{T}_B$ and let (Π_B, Π_I, Θ) be a theory-modular encoding of Π .

Theorem: Π is satisfiable **iff** (Π_B, Π_I, Θ) is satisfiable.

Decidability: the decidability of CHC-SAT modulo $\mathcal{T}_B$ is not compromised.

The Mosaic network

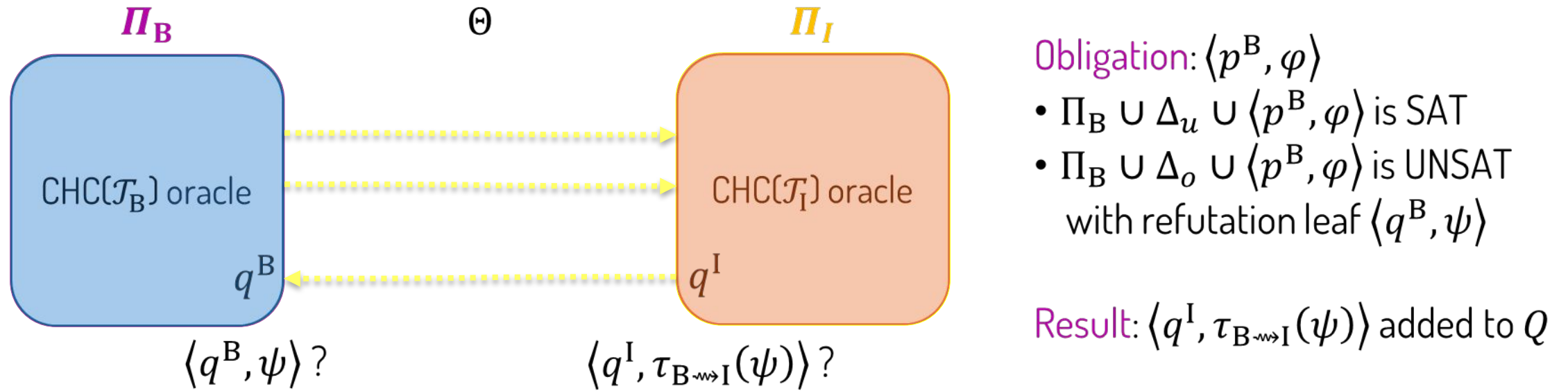


MosaicSolve: state

State: $Q \parallel \Delta_u \parallel \Delta_o$

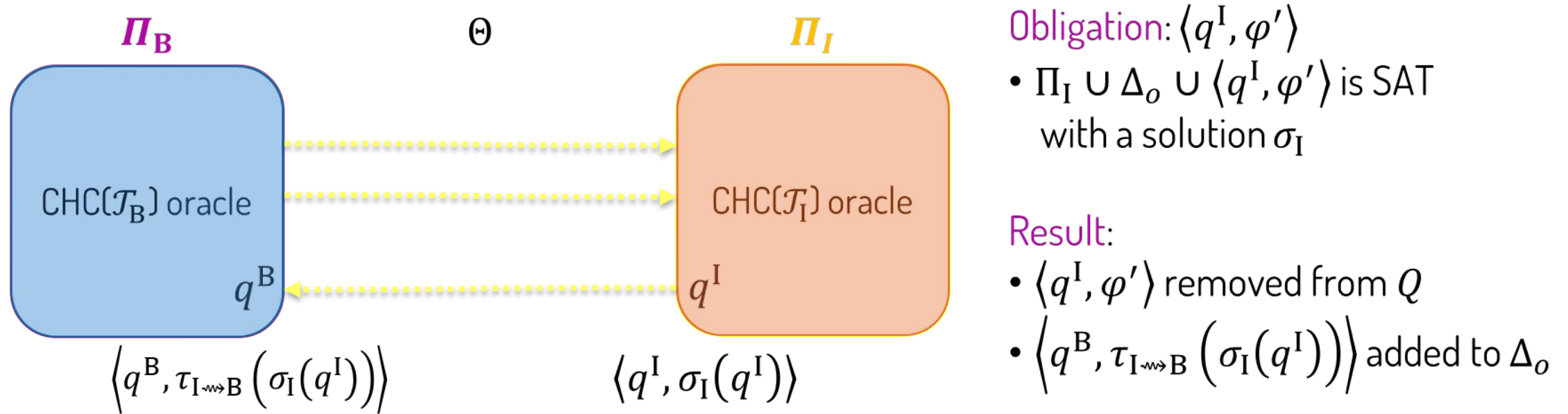
- Q : **proof obligations** $\langle p, \varphi \rangle$ for a predicate p .
Is there a tuple derivable for p satisfying φ ?
- Δ_u : **under-approximation lemmas** $\langle p, \psi \rangle$ for the external semantics of an interface target predicate p .
Find real counterexamples.
- Δ_o : **over-approximation lemmas** $\langle p, \psi \rangle$ of the same.
Block infeasible counterexamples + detect convergence.

MosaicSolve: moves



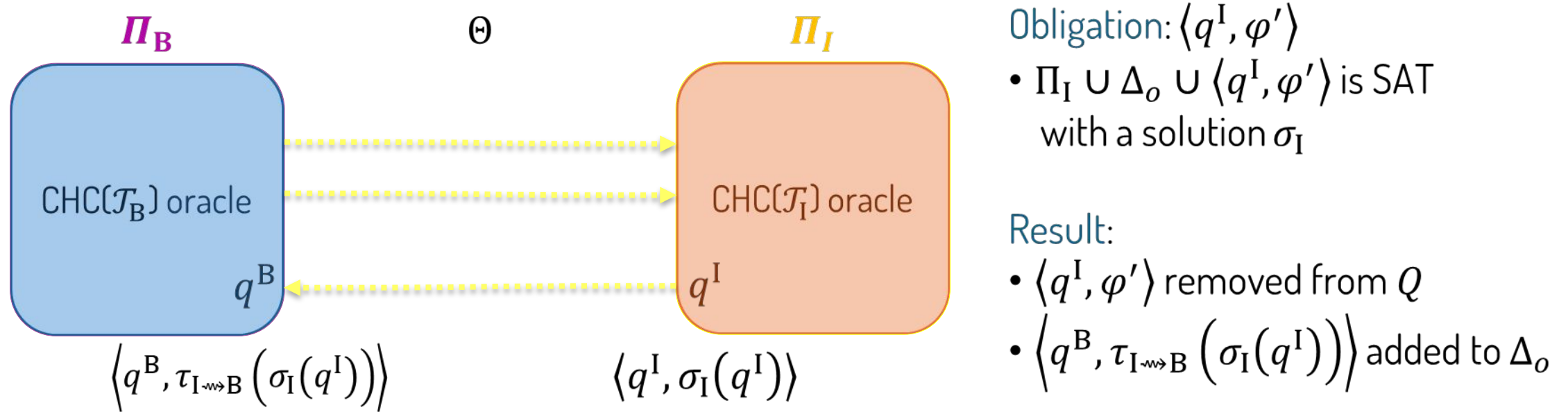
- **Query:** generates a new obligation when an existing one cannot be resolved.

MosaicSolve: moves



- **Query:** generates a new obligation when an existing one cannot be resolved.
- **NegAnswer:** discharges an obligation if it can be answered negatively using the current over- approximations.

MosaicSolve: moves



- **Query**: generates a new obligation when an existing one cannot be resolved.
- **NegAnswer / PosAnswer**: discharges an obligation if it can be answered negatively / positively using the current over- / under-approximations.
- **Sat / Unsat**: terminate when the initial obligation can be resolved.

MosaicSolve: worked run

Q	Δ_u	Δ_o
$\langle q^B, \neg((a \oplus^B b) <_s^B 0000) \rangle$		$\langle q^B, a >_s^B 0000 \wedge b <_s^B 0000 \rangle$
$\langle q^I, a' = 7 \wedge b' = 0 \rangle$		

Π_I :

$$\langle 0 \rangle \quad (x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0) \rightarrow p^I(x', a', b')$$

$$\langle 1 \rangle \quad (p^I(x', a', b') \wedge a' < x') \rightarrow p^I(x', a' + 1, b' - 1)$$

$$\langle 2 \rangle \quad (p^I(x', a', b') \wedge \neg(a' < x')) \rightarrow q^I(a', b')$$

$$\langle q^I, a' = 7 \wedge b' = 0 \rangle? \quad \times$$

$$\sigma_I(q^I(a', b')) := a' > 0 \wedge b' < 0$$

Π_B :

$$\langle 3 \rangle \quad q^B(a, b) \rightarrow ((a \oplus^B b) <_s^B 0000)$$

$$\langle q^B, a = 0111 \wedge b = 0000 \rangle?$$

SAT 

Soundness and termination

Let Π be a set of CHCs modulo $\mathcal{T}_B$ and let (Π_B, Π_I, Θ) be a theory-modular encoding of Π .

Theorem (soundness): If MosaicSolve returns SAT (resp. UNSAT), then (Π_B, Π_I, Θ) is satisfiable (resp. unsatisfiable).

Termination: MosaicSolve is guaranteed to terminate if each invocation of the underlying CHC satisfiability oracles terminates.

Evaluation: setup

Implementation

- Prototype on top of Z3
- Spacer instantiates both CHC-SAT oracles

Benchmarks

- 11 small programs that originate from the "Bit Twiddling Hacks" collection [Anderson, Stanford], composed with simple arithmetic loops
- Benchmarks evaluated for increasing bit-widths (timeout: 2 hours)

Configurations

- BV: all CHCs in the bit-vector fragment
- IA: all CHCs translated to the integer fragment
- BV+IA: our theory-modular reasoning

Evaluation: results

BV+IA achieves the largest bit-width on most benchmarks, often by a substantial margin

	BV		IA		BV+IA	
	W	Time [s]	W	Time [s]	W	Time [s]
abs-ge						
abs-sum						
cond-neg						
cond-neg-diff						
max-inv						
max-sum						
opp-signs-diff						
swap						
swap-sum						
turn-off-rm						
turn-on-lsb						

Performance of BV, IA, and BV+IA.

Maximal bit-width (W) and runtime.

Bold indicates the largest W.

Conclusion



Mosaic: Theory-Modular CHC Satisfiability using $\mathcal{T}_B$ and $\mathcal{T}_I$

Key ingredients

- **Theory transformations:** translations across theories preserving the represented bit-vector semantics
- **Theory-modular encoding:** decomposition into (Π_B, Π_I, Θ)
- **Theory-modular solving:** coordinated reasoning across fragments

Result: substantial scalability improvements on bit-precise verification benchmarks

Related and Future Work

- Abstraction Refinement for Theory-Modular CHC Satisfiability
- Extending theory-modular reasoning to additional background theories (LIA + NIA)

Questions?

Application 1 of CHCs

Condition Synthesis Realizability via Constrained Horn Clauses

Bat-chen Rothenberg, Orna Grumberg, Yakir Vizel, Eytan
Singher

NFM 2023

Condition Synthesis

- Receives as input a **partial program**, where conditions are missing in conditional branches (e.g., if/while statements), and a **specification**
 - As **assertions** in the code
- A **solution** to this problem is a set of conditions to fill-in the holes such that the resulting program **meets the specification**.
- **Motivation:** automated program repair

Condition Synthesis Running Example

```
sum_abs(x,y):
```

```
  sum = 0
```

```
  if (x >= 0)
```

```
    sum = sum + x
```

```
  else
```

```
    sum = sum - x
```

```
hole → if ( ?? )
```

```
    sum = sum + y
```

```
  else
```

```
    sum = sum - y
```

```
  assert (sum >= 0) ← specification
```

The problem is
realizable!



possible solution: $y \geq 0$

Our Algorithm

- We present a novel algorithm, called **COSYN**, for determining the **realizability** of a condition synthesis problem, with an emphasis on **proving unrealizability efficiently**.
- Cosyn can handle programs with **multiple holes**.
- Cosyn can handle programs with **loops**.

Doomed initial states

- An initial state is called *doomed* if it eventually reaches an error state along *every* run of the program.
- For a doomed initial state σ , there is no way to make the program safe by forcing σ (via conditions) to follow one computation or another. *It will lead to a failure anyway.*
- The problem is **unrealizable** iff there exists a **doomed initial state**

Condition Synthesis Running Example

$x=0, y=-3$

$x=0, y=-3, \text{sum}=0$

$x=0, y=-3, \text{sum}=0$

$x=0, y=-3, \text{sum}=0$

$x=0, y=-3, \text{sum}=-3$

$x=0, y=-3, \text{sum}=-3$

$x=0, y=-3, \text{sum}=-3$



sum_abs(x,y):

sum = 0

if (x >= 0)

sum = sum + x

else

sum = sum - x

if (??)

sum = sum + y

else

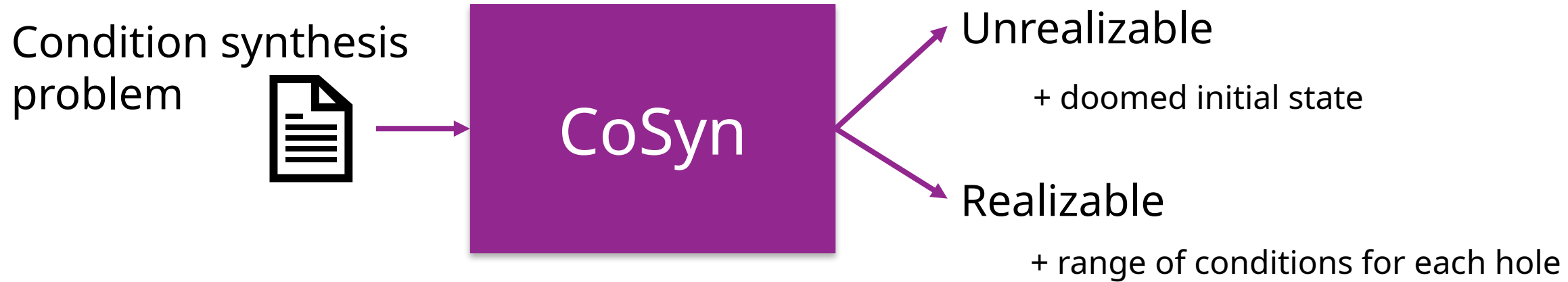
sum = sum + y

assert (sum >= 0)

Problem is
unrealizable!



Correctness Evidence



Condition Synthesis Running Example

```
sum_abs(x,y):
```

```
    sum = 0
```

```
    if (x >= 0)
```

```
        sum = sum + x
```

```
    else
```

```
        sum = sum - x
```

```
        if ( ?? )
```

```
            sum = sum + y
```

```
        else
```

```
            sum = sum - y
```

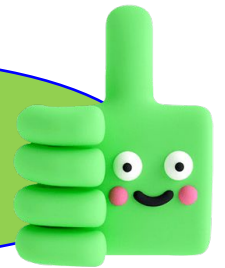
```
    assert (sum >= 0)
```

possible range of solutions:

$\backslash\psi_f$: $\text{sum} - y < 0 \ \& \ \text{sum} \geq 0$

$\backslash\psi_t$:

Problem is
realizable!



Reduction to CHC

- The problem is **unrealizable** iff there exists a **doomed initial state**.
- To check for the existence of a doomed initial state, COSYN uses a **reduction** to **Constrained Horn Clauses (CHC)**.

End of lecture 3